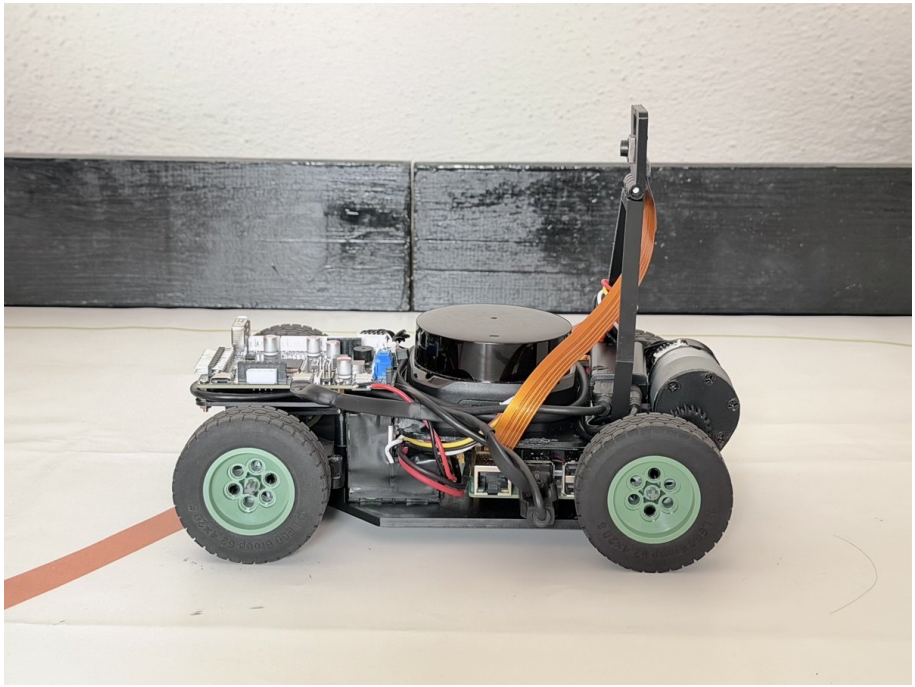
ENGINEERING JOURNAL

Team CSS

WRO Future Engineers 2026

International Final edition · Puerto Rico, December 2026

Querétaro, Mexico



*The international competition vehicle, **"The Maker"** (design v7)*

Vehicle	"The Maker", design v7 (international). Design v6 placed 2nd at the 2026 Mexico National Final
Team members	Christian Gael Centeno Velez (Captain) · Sebastian Esquivel Mondragon
Coach	Manuel Alejandro Cardoso Duarte
Dimensions / Mass	238.5 × 157.7 × 180 mm · 1.087 kg
Drive / Steering	Rear-wheel drive with differential, JGB37-520 30:1 encoder motor · Front Ackermann, HX-35H bus servo
Compute	Raspberry Pi 5 + Hailo-8 · Hiwonder ROS Robot Control Board (STM32F407)
Sensors	Slamtec RPLidar S2L · Raspberry Pi Camera Module 3 Wide · IMU on the Hiwonder board
Power	3 × Panasonic NCR18650B in series (3S, 10.8 V nominal) with 3S BMS
Repository	github.com/alex309-duarte/WRO_Future_Engineers_Queretaro_CSS (release v2.0.0)

How to read this journal

This is the **international edition** of our engineering journal. The national edition (September 2026, design v6) is preserved in the repository as release **v1.0.0**. This edition keeps the full history, because the reasons behind the national design are still the reasons behind most of design v7, and adds what we changed for the international final, why, and how we tested it.

Like the national edition, it is organised around the five evaluation criteria in Appendix C of the 2026 Future Engineers rules:

Rubric criterion	Where it is addressed
1. Mobility and Mechanical Design	Section 2 - steering, drivetrain, motor sizing (v6 and v7), differential, chassis, design evolution v1-v7
2. Power and Sensor Architecture	Section 3 - power budgets (v6 and v7), failures, LiDAR decisions in both directions, placement, calibration
3. Software Architecture and Obstacle Strategy	Section 4 - threading, hardware abstraction layer, perception, control, strategies, vision model, emergency stop
4. Systems Thinking and Engineering Decisions	Section 5 - tradeoff register, risk register, subsystem coupling
5. Reproducibility and GitHub Quality	Sections 6-8 - testing workflow, dated logbook, repository map, interfaces, versions

Design numbers

We number every vehicle we have built since 2025, so **v1-v7 are vehicle designs**, not document versions: v1 and v2 are the 2025 robots, v3-v6 the 2026 national iterations, and **v7** is the international vehicle. Repository releases use their own numbers: release v1.0.0 is design v6 and release v2.0.0 is design v7.

What changed since the national edition

Subsystem	Design v6 (national)	Design v7 (international)	Section
Motor controller	LEGO SPIKE Prime Hub	Hiwonder ROS Robot Control Board (STM32F407)	4.9
Drive motor	LEGO Large Angular Motor, 175 rpm	JGB37-520 30:1, 320 rpm, Hall encoder	2.3
Rear differential	v6 differential	New differential, same 20:28 ratio	2.2
Steering actuator	LEGO Small Angular Motor	HX-35H serial bus servo	2.1
Heading	SPIKE Hub IMU	Hiwonder board IMU (20 Hz)	4.4
Battery	4 × NCR18650B + Geekworm X1203	3 × NCR18650B in series (3S) + BMS	3.1
LiDAR	Oradar MS200k	RPLidar S2L (brought back)	3.4
AI accelerator / model	Hailo-10H, 501-image model	Hailo-8, 23,938-image model	3.6, 4.8
Safety	No emergency stop (open risk)	Signal-based emergency stop	4.10
Chassis	Custom, SolidWorks	Modified for the new parts and lowered	2.4

Every constant quoted in this journal is the value present in the source code in our public repository, not a design target.

1. Design philosophy and season constraints

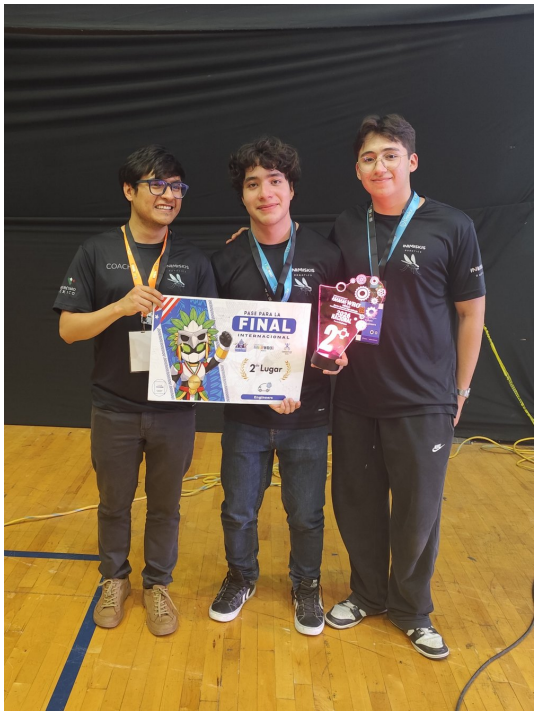
We did not start this season from a blank page. We finished the 2025 season with a working Future Engineers vehicle that placed 2nd nationally, and our starting decision was whether to iterate on it or rebuild. We chose to keep the concept - Ackermann-steered, LiDAR-led, LEGO SPIKE actuation - and rebuild everything else, because our post-season review identified three problems that were structural rather than tuning issues:

- **Latency.** The 2025 codebase was Python. Adding a higher-rate camera and LiDAR made the control loop the bottleneck, not the sensors.
- **Mass and packaging.** The 2025 vehicle came in barely under the 1.5 kg limit, with a high centre of gravity and loosely packaged electronics.
- **Mechanical inconsistency.** The custom double-Ackermann steering we built in 2025 gave a tighter turning radius but did not behave the same way twice.

Those findings became the season's design goals: **cut latency, cut mass, and trade peak capability for repeatability wherever the two conflict.**

1.1 Goals for the international final

After the national final (2nd place with design v6) we kept the same principle and added one goal: **more speed without losing repeatability.** The LEGO drivetrain reached only 0.365 m/s (Section 2.3). For the international final we replaced the LEGO SPIKE actuation with a faster encoder motor, a bus servo and a dedicated motor-control board, and we required that the mission logic that placed 2nd nationally be reused rather than rewritten (Section 4.9), because there were less than eight weeks between the national final and the repository deadline for the international final (10 November 2026).



Team CSS with coach Manuel Alejandro Cardoso Duarte after the 2026 Mexico National Final: 2nd place and qualification for the International Final.

1.2 Constraints we designed against

Constraint	Source	Limit	v6 value	v7 value	How it shaped the design
Maximum dimensions	Rule 11.1	300 × 200 × 300 mm	240 × 160 × 250 mm	238.5 × 157.7 × 180 mm	Dense packaging; Hailo-8 chosen partly to reduce size
Maximum mass	Rule 11.2	1.5 kg	1.006 kg (67 %)	1.087 kg (72 %)	Mass freed in v7 allowed the S2L back
One driving axle, one steering actuator	Rule 11.3	-	RWD + front Ackermann	RWD (differential) + front Ackermann	Ruled out AWD and two drive motors
No wireless during rounds	Rule 11.10	-	Ethernet only	Ethernet only	See Section 5.2
Track wall height	Game field	100 mm	LiDAR at 60-80 mm	Chassis lowered for the LiDAR	Hard constraint - Section 3.5
Round time limit	Rules 9.1 / 9.2	3 minutes	-	-	Minimum average speed for 3 laps
Time to the international repository deadline	Calendar	< 8 weeks (19 Sep - 10 Nov)	-	-	Reuse the national mission code (4.9)
Budget	Team	-	$23,679.82 MXN (design v7 BOM; design v6: $33,018.73)	Reused Pi 5, camera, tyres, cells and button from v6	No spare vehicle; parts carried over where possible

The mass figure deserves emphasis. Going from barely under 1.5 kg to 1.006 kg in v6 was the root cause of half the changes in Section 2. In v7 the same margin paid off again: removing the SPIKE Hub, the X1203 board and one battery cell left room for the heavier RPLidar S2L (185 g) without approaching the limit.

The parking bay is defined as 1.5× our own vehicle length, so making the robot shorter makes the bay proportionally shorter too. Shrinking the chassis does not make parking easier; it holds the difficulty constant while shrinking our margin for error in absolute millimetres.

2. Mobility and mechanical design

2.1 Steering: why we went backwards on purpose

In 2025 we designed and built a custom **double-Ackermann** linkage - both axles steering - to get a tighter turning radius. It worked, and on a good run it was better than a conventional layout. The problem was that it was not consistent: the additional linkage introduced play, and the steering did not return to the same neutral point reliably across runs.

For 2026 we replaced it with a **regular Ackermann** linkage steering the front axle only, reaching a maximum steer angle of **45.44°** in design v6. The design v7 linkage, now driven by a bus servo, reaches **45.9°** in the CAD, so the turning radius is the same as in v6; in software the steering is limited to ±35°.

> **We chose regular Ackermann instead of double-Ackermann because** the double-Ackermann's advantage was a smaller turning radius, but our failure mode was never "the corner was too tight to make" - it was "the robot did not end the corner pointing where we expected." The regular Ackermann completes every turn the WRO track requires with margin, and it removes a mechanical error source from every corner of a 12-corner run.

The Ackermann condition - inner and outer wheels tracing concentric arcs about a common centre - keeps the front tyres rolling instead of scrubbing. Scrubbing costs traction mid-corner, and it makes the effective wheelbase behave differently at different steer angles, which is exactly the non-repeatability we were trying to design out.

Design v6 actuator

Steering was driven by a LEGO SPIKE Small Angular Motor (45607), replacing the Medium Angular Motor used previously. This was a consequence of the mass reduction: with a lighter chassis the required steering torque dropped enough that the smaller motor became sufficient.

Design v7 actuator: HX-35H serial bus servo

With the SPIKE Hub removed (Section 4.9), the steering needed an actuator the Hiwonder board can command. We use a **Hiwonder HX-35H** serial bus servo on the same Ackermann linkage.

HX-35H (Hiwonder)	Value	Why it matters for us
Voltage	9-12.6 V	Runs directly from the 3S battery, no extra regulator
Stall torque	35 kg·cm at 11.1 V	Large margin over the linkage load
Speed	0.18 s/60° at 11.1 V	Fast steering response at battery voltage
Range / command	0-240°, 0-1000 (0.24° per unit)	Matches kGradosPorUnidadServo = 0.24 in the code
Feedback	Position, temperature, voltage	Start-up check of the servo position
Stall current / mass	3 A / 52 g	Counted in the peak-current analysis (3.1)

> **We chose a serial bus servo instead of a PWM hobby servo because** the bus servo closes its own position loop, reports its position, and shares the serial bus with the board, so the board can both command and verify the steering. It runs at battery voltage, and its torque margin means the steering reaches its commanded angle at the same speed regardless of tyre grip on the mat.

Safety in software: every steering command passes through one function (*set_steering*) that clamps to **±35°** from centre (35 / 0.24 = 145 servo units each side, positions 605-895), the program refuses to move the servo at start-up if it is outside the allowed range, and the emergency stop releases the servo torque so it cannot push the linkage against an end stop.

2.2 Drivetrain: AWD to RWD, and a new differential

The 2025 vehicle was all-wheel drive. The 2026 vehicles are rear-wheel drive.

> **We chose RWD instead of AWD because** AWD required a front differential and driveshaft to carry torque past the steering axle. That hardware cost mass, volume in exactly the region where the Ackermann linkage lives, and one more set of mechanical tolerances. On a flat, high-grip competition mat the traction AWD buys is not the limiting factor.

Design v7: rear differential

Design v7 uses a **new rear differential**: the LEGO 65414 differential housing with the 65413 28-tooth bevel gear in a custom mount (CAD: Diferencial.SLDASM). The gear ratio stays 20:28, so the speed and torque calculation of Section 2.3 and the distance constants of the missions keep their meaning.

> **We replaced the v6 differential instead of adapting it because** its shape and size did not fit the modified chassis and the new JGB37 motor well; the new one fits the structure and the motor mount better. We keep a differential at all because in a corner the outer rear wheel travels farther than the inner one: without it one wheel must slip, which wastes traction, wears the tyres and makes the encoder-based distance less accurate.

2.3 Motor sizing: the calculation we did before buying anything

Design v6: LEGO Large Angular Motor

We sized the traction motor against a dynamic torque budget rather than picking a motor and hoping. Target: reach cruising speed in t = 0.5 s, implying a = 0.73 m/s². Inputs: m = 1.006 kg, r = 0.0312 m, C_{rr} = 0.03, gear reduction 20:28 (R = 1.4), efficiency 0.85.

- Acceleration force F_{acc} = m·a = 0.7344 N; rolling resistance F_{rr} = C_{rr}·m·g = 0.2961 N; total 1.0305 N
- Wheel torque T_w = 0.03215 N·m; motor torque T_m = T_w/(η·R) = 0.02702 N·m (2.70 N·cm); with a 1.25× margin 3.38 N·cm

Metric (LEGO 45602)	Value	Interpretation
Load vs max-efficiency torque (8.00 N·cm)	33.8 %	Motor runs well inside its efficient band
Load vs stall torque (25.00 N·cm)	10.8 %	No realistic stall risk
Loaded top speed	0.365 m/s	Reached in 0.5 s at the design acceleration
Torque reserve	66.2 %	Absorbs perturbations without dropping speed

Design v7: Hiwonder JGB37-520, 30:1

For the international final we wanted more speed while keeping the same principle: a motor that works far below its limit holds its speed when the load changes. We chose the Hiwonder JGB37-520R30-12V Hall-encoder gear motor.

> **We chose the JGB37 instead of keeping the LEGO motor because** the SPIKE motor was too slow for the level we expect at the international final, where every team is the best of its country. We chose this motor in particular because we already had a similar Hiwonder motor and were familiar with it, and its speed and torque cover our needs with margin (calculation below).

JGB37-520R30-12V (Hiwonder)	Value
Rated voltage / gear ratio	12 V / 30:1
No-load speed	320 rpm
Rated speed / torque / current	280 rpm / 1.2 kg·cm / 0.36 A
Stall torque / current	5.8 kg·cm / 3.2 A
Encoder	Hall, AB quadrature, 11 pulses per motor revolution → 11 × 30 × 4 = 1320 counts per output revolution (matches the 1320 ticks/rev we measured)
Mass	152 g

Same method as v6, with the measured v7 mass **m = 1.087 kg**, a = 1.3 m/s², r = 0.0312 m (LEGO 62.4 × 20 S tyres), the same 20:28 gear reduction as v6 (R = 1.4), η = 0.85, and the 3S nominal voltage of 10.8 V:

- F_{acc} = 1.087 × 1.3 = 1.413 N; F_{rr} = 0.03 × 1.087 × 9.81 = 0.320 N; T_w = 1.733 × 0.0312 = 0.0541 N·m

- T_m = 0.0541 / (0.85 × 1.4) = 0.0454 N·m (0.46 kg·cm); with the 1.25× margin 0.0568 N·m (0.58 kg·cm)
- No-load speed at 10.8 V: 320 × 10.8/12 = 288 rpm; loaded: 288 × (1 − 0.0454/0.569) = 265 rpm; wheel: 265 / 1.4 = 189 rpm

Metric (JGB37 30:1)	Value	Interpretation
Load vs rated torque (1.2 kg·cm)	39 % (48 % with margin)	Inside the continuous rating
Load vs stall torque (5.8 kg·cm)	8 %	No stall risk in normal driving
Loaded top speed at 10.8 V	0.62 m/s	1.7× design v6
Time to top speed	0.48 s	Meets the 0.5 s target
Same check at the 1.5 kg rule limit	53 % of rated torque, 0.60 m/s	The motor still has margin if mass is added

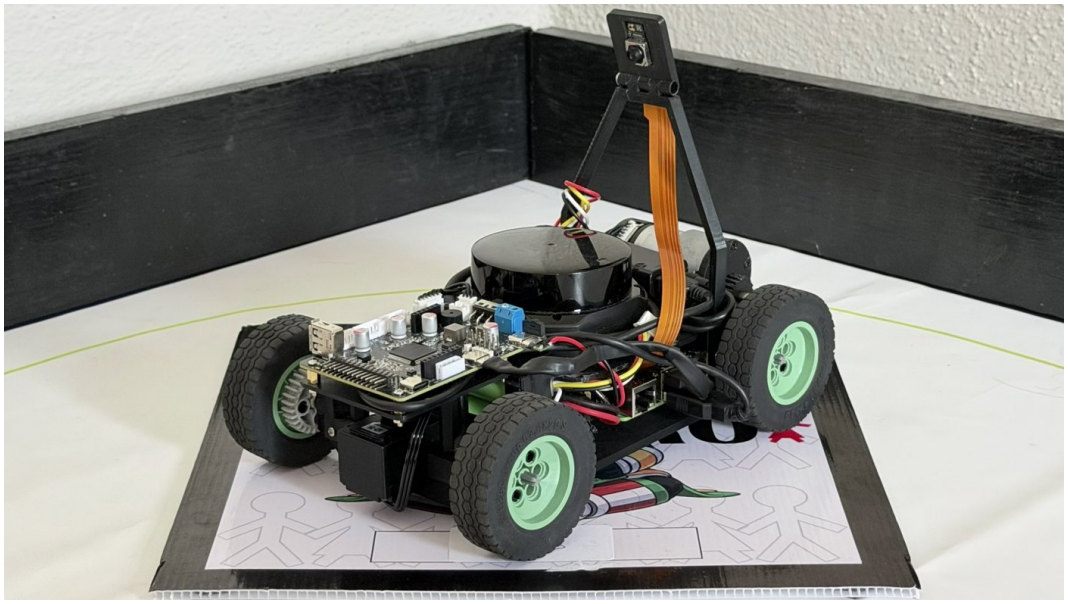
Speed set-point and battery voltage. The motor's speed falls with battery voltage (no-load 320 rpm at 12 V, 288 rpm at 10.8 V). If the speed set-point for speed = 100 (kRpsVelocidad100) is above what the motor can reach on a partly discharged pack, the speed loop saturates and the real speed depends on the battery charge. A lower set-point (about 4.0 rev/s) gives the loop more headroom from a full (12.6 V) to a nearly empty (≈10.5 V) pack, which reduces the speed change across a session. This follows the same reasoning as running the v6 motor at a third of its capacity. Measured on the track (Section 6.1): 0.580 m/s with a full battery and 0.524 m/s with a low one at 5.7 rev/s, and 0.539 / 0.506 m/s at 4.0 rev/s.

2.4 Chassis and centre of gravity

The chassis is a fully custom design, modelled in SolidWorks and printed on a Creality K1C. The 2026 revision repackages the electronics densely and moves the heavy parts lower in the frame. The objective is lateral weight transfer: in a corner, transfer scales with CG height; less transfer keeps the inside front wheel loaded, which keeps the Ackermann geometry doing what it was designed to do.

Design v7 changes

The v7 chassis is a modification of the v6 design, made for two reasons. First, to fit the new components cleanly: the Hiwonder board, the JGB37 motor, the HX-35H servo, the 3S battery with its BMS and the RPLidar S2L have different shapes and mounting points from the SPIKE Hub, the LEGO motors and the MS200k (new parts include a combined Hiwonder board / LiDAR support and the differential). Second, to sit **as low to the ground as possible**, so the LiDAR scanning plane stays well below the top of the 100 mm walls (the failure of Section 3.5) and the centre of gravity stays low at the higher speed.

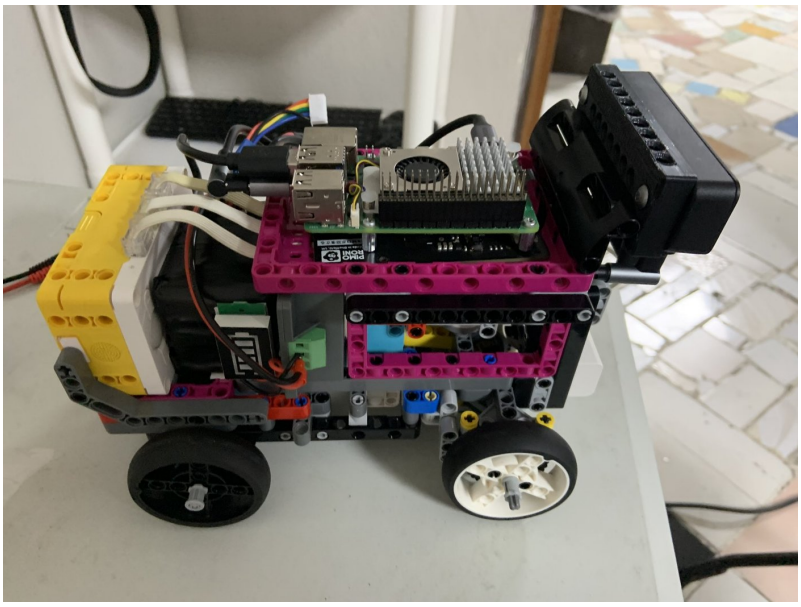


Design v7, isometric view: Hiwonder controller at the front, RPLidar S2L in the centre, camera mast and JGB37 drive motor at the rear. SolidWorks assembly in models/cad/v7_assembly/.

2.5 Design evolution: seven vehicles in two seasons

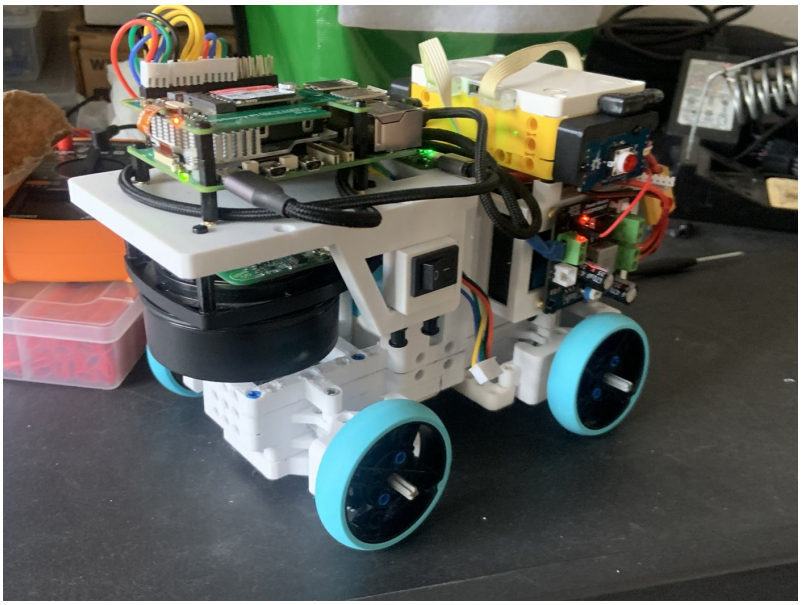
Our international vehicle is the **seventh** distinct design the team has built across two seasons. There is a deliberate discontinuity between v2 and v3; within 2026 the progress was incremental, and v7 is the largest step since v3.

2025 season - v1 and v2



v1 - hybrid LEGO Technic chassis, regional competition (Querétaro). Yellow SPIKE Prime hub, Raspberry Pi on a Technic deck, OAK-D Lite camera at the front.

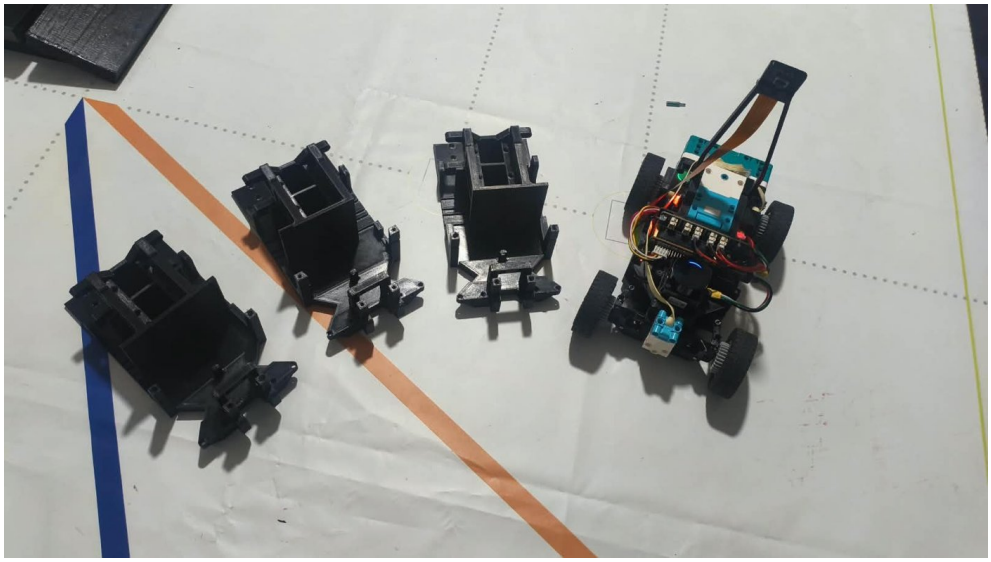
Our first Future Engineers vehicle was built partly from LEGO Technic structural elements. It was fast to iterate, but component placement was dictated by the Technic hole grid, the electronics sat high above the axles, and the assembly had play at every beam connection.



v2 - fully custom 3D-printed chassis, national competition (2nd place nationally). RPLidar S2L on the front deck, double-Ackermann steering, AWD.

For the 2025 national competition we designed a fully custom, 3D-printed chassis. It carried the double-Ackermann steering and the AWD drivetrain to 2nd place nationally. Its weaknesses are the three findings listed in Section 1: heavy (barely under 1.5 kg), high centre of gravity, and mechanically inconsistent steering.

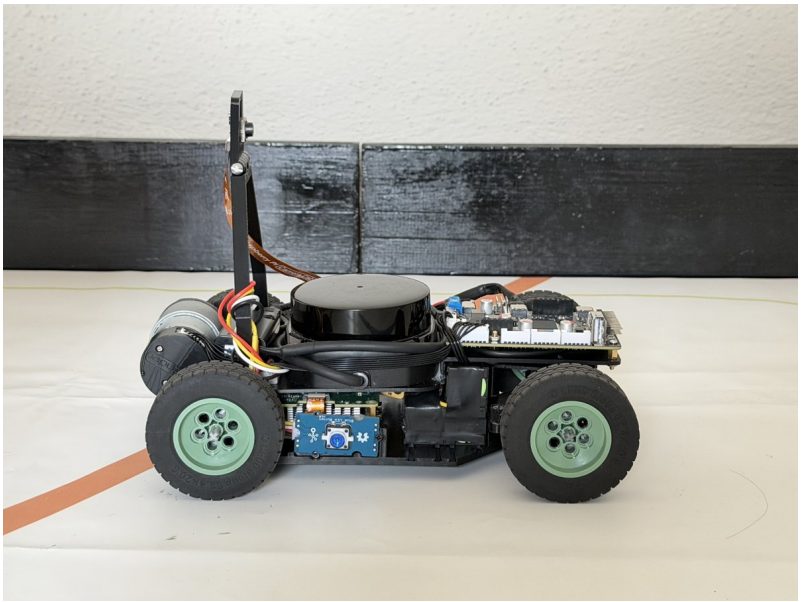
2026 season - a clean-sheet redesign (v3), refinements (v4-v6), and the international vehicle (v7)



The 2026 national iterations side by side: v3, v4 and v5 as bare printed frames, with v6 assembled at the right.

#	Iteration	Focus of this revision
v3	2026 clean sheet	New layout: RWD, regular Ackermann, battery and Pi dropped low in the frame
v4	Packaging	Component placement tightened around the new layout
v5	Mounting	Mounting and serviceability fixes, including the LiDAR mount revision of Section 3.5
v6	National final	Final national geometry - 2nd place at the 2026 Mexico National Final (release v1.0.0)
v7	International final	Hiwonder board, JGB37 motor, differential, HX-35H servo, 3S battery, RPLidar S2L, Hailo-8; chassis modified and lowered (release v2.0.0)

We chose a clean-sheet redesign instead of iterating v2 because the three problems we had were all *layout* problems. **For v7 we chose to modify v6 instead of starting again because** the v6 layout was right - the changes were in the actuators, power and sensors, not in the concept. Knowing which mode you are in matters: retrofitting a broken layout wastes time, and clean-sheeting a working one throws away tuning.



*Design v7, right side. The full set of six views is in the repository (v-photos/*_v7.JPG).*

3. Power and sensor architecture

3.1 Power

Design v6: 4 × NCR18650B + Geekworm X1203

Design v6 used four Panasonic NCR18650B cells (3400 mAh each) in a 4P configuration at 3.7 V nominal (13.6 Ah, 50.32 Wh in total) feeding a Geekworm X1203 UPS/boost board that supplied the 5.1 V for the Raspberry Pi 5. The LEGO motors were powered by the SPIKE Hub's own battery.

Component (v6)	State	Power
Raspberry Pi 5	High CPU / OS load	9.0 W
Hailo-10H AI HAT	Continuous inference	4.0 W
Oradar MS200k LiDAR	Scanning	1.5 W
Grove modules	Relay pulses + LED button	0.1 W
Total at 5.1 V		14.6 W

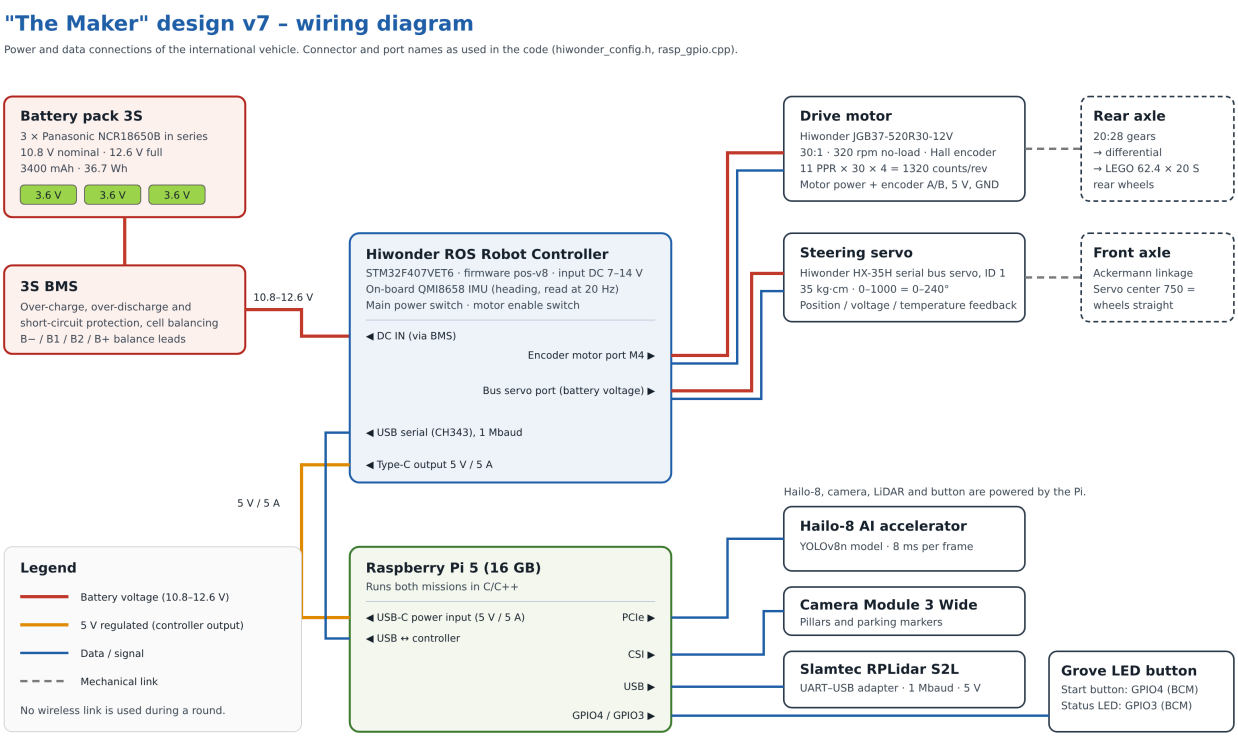
Theoretical worst case 50.32 Wh / 16.22 W = 3.1 h; measured 6.8 h, because sustained peak CPU and inference load does not occur in a real round.

Design v7: one 3S pack for everything

Design v7 uses **three NCR18650B cells in series (3S)**: 10.8 V nominal, 12.6 V full, 3400 mAh, 36.7 Wh, protected by a 3S BMS. The pack feeds the Hiwonder board, which drives the motor and servo at battery voltage.

> **We chose a single 3S pack instead of keeping the v6 pack plus a second battery for the motors because** the JGB37 motor and the HX-35H are 12 V parts and the Hiwonder board is designed for a 12 V supply. One 3S pack gives that voltage directly, removes the X1203 boost stage and the SPIKE battery, and means one battery to charge and check. We kept the NCR18650B cells because they solved the brownouts of Section 3.2.

The pack feeds the **Hiwonder ROS Robot Controller** (STM32F407VET6, input 7-14 V). The controller drives the motor and the bus servo at battery voltage, and its independent **5 V / 5 A Type-C output powers the Raspberry Pi 5**, which in turn powers the Hailo-8, the camera, the LiDAR and the start button. The Raspberry Pi 5 needs 5 V / 5 A to run its USB ports at full current, which matters because the LiDAR is powered from USB. The Pi and the controller communicate over a USB serial link (UART, 1 Mbaud).



Design v7 wiring diagram: battery voltage (red), regulated 5 V (orange), data (blue) and mechanical links (dashed). Also in the repository as schemes/hardware/wiringSchematics_v7.png.

Component (v7)	Supply	State	Estimate
Raspberry Pi 5	5 V	High CPU load	9.0 W
Hailo-8	5 V	Continuous inference (typical, manufacturer figure)	2.5 W
RPLidar S2L	5 V	10 Hz scanning, 500 mA typical (datasheet)	2.5 W
Grove LED button	5 V		0.1 W
5 V subtotal		≈ 90 % regulator efficiency	14.1 W → 15.7 W from the pack
JGB37 drive motor	Battery	Accelerating at 1.3 m/s² (≈ 0.35 A)	3.8 W
HX-35H servo	Battery	Holding / small corrections (≈ 0.1 A)	1.1 W
Hiwonder controller logic	Battery	Estimate	0.5 W
Total		≈ 1.95 A at 10.8 V	≈ 21 W → runtime ≈ 1.7 h

Peak current is the v7 power risk. The motor stall current is 3.2 A and the servo stall current is 3 A. In the worst case (both stalled while the 5 V side draws about 1.45 A) the pack would supply about 7.7 A, above the NCR18650B's 4.875 A maximum continuous discharge. In a 3S pack every cell carries the full current; there is no parallel branch to share it. It is a short-duration case, but it is the same kind of transient that caused the brownouts of Section 3.2, so we calculated it from the Panasonic datasheet (internal resistance ≤ 45 mΩ per cell).

Quantity (calculated)	Calculation	Result
Average pack current	21 W ÷ 10.8 V	≈ 1.9 A (40 % of the rating)
Start from standstill	motor stall 3.2 A + 5 V side 1.45 A + servo 0.1 A	≈ 4.75 A (97 %), transient only
Worst case	motor and servo stalled + 5 V side	≈ 7.7 A (157 %)
Voltage sag	I × 0.135-0.27 Ω (3 cells, with margin for DC resistance, BMS, wiring)	0.6-1.3 V at 4.75 A; 1.0-2.1 V at 7.7 A
Lowest board input	low pack 9.9 V minus worst-case sag	≈ 7.8 V, above the board's 7 V minimum
Energy per round	21 W × 121 s (Obstacle) / 48 s (Open)	0.71 Wh (2 %) / 0.28 Wh (0.8 %)

Even the worst case keeps the Hiwonder board inside its 7-14 V input range, so its regulated 5 V / 5 A output to the Raspberry Pi is not interrupted. The risk is tracked in the register of Section 5.2.

3.2 Failure: brownouts under acceleration (June 2026)

Symptom. With the Raspberry Pi 5 and the Hailo accelerator both active, the system would shut down abruptly during hard acceleration. **Investigation.** The pack still showed plenty of charge; the correlation with acceleration pointed at a transient: the traction motor's current spike was pulling the rail down far enough to reset the Pi through the high internal resistance of generic 18650 cells. **Fix.** Panasonic NCR18650B cells, which hold a much flatter discharge curve under load, and the X1203 board, which both regulates and buffers the rail.

> **What we took from it.** A power system is not specified by capacity alone. Internal resistance and discharge-curve flatness were what actually mattered. In v7 this is why we kept the same cells and why the Raspberry Pi is fed from a dedicated regulated 5 V / 5 A output instead of directly from the pack.

3.3 LiDAR, July 2026: RPLidar S2L to Oradar MS200k

During early testing the RPLidar S2L returned distorted and imprecise readings when the vehicle passed very close to a track wall - exactly the condition our wall-following logic depends on most. The S2L was also heavier (185 g) at a time when we were fighting for every gram. We replaced it with the Orbbec Oradar MS200k, which is lighter and handled short-range reflections cleanly in our tests.

> **We chose the MS200k instead of keeping the S2L because** the readings were worst in the operating region we actually live in, and because of the mass. Section 3.4 explains why this conclusion was wrong.

3.4 LiDAR, October 2026: back to the RPLidar S2L

For design v7 we returned to the S2L. Both original reasons had changed:

- **Mass.** Removing the SPIKE Hub, the X1203 board and one battery cell freed enough mass for the 185 g S2L.
- **Readings.** With time to test it properly, the S2L proved accurate and reliable. The distorted readings in July came from our own integration, not from the sensor: we had reused the driver program written for another Slamtec LiDAR (the RPLidar A1) and, implementing it in the last days before the change, never properly adapted it to the S2L.

RPLidar S2L (Slamtec datasheet)	Value
Range	0.05-18 m (white), 0.05-8 m (black)
Sample rate / scan rate	32 kHz / 10 Hz
Angular resolution	0.1125° (3,200 points per turn)
Accuracy / resolution	±50 mm typical / 13 mm
Supply	5 V, 500 mA typical (1.5 A at start-up)
Interface / mass	UART 1 Mbaud / 185 g

> **We chose the S2L instead of keeping the MS200k because** once its driver was properly adapted - with enough time and attention, not at the last minute - the S2L gave us reliable readings, more points per turn and a longer range. In our tests it detected the wall in 98.4 % of frames against 96.2 % for the MS200k, with lower noise at every distance (Section 6.1). **What we took from it:** we had judged the sensor by an untested integration. A component should be ruled out only after its integration has been tested on its own, with enough time to separate a sensor problem from a software problem.

The driver is selected at build time: the S2L is the default in the v7 build, and the MS200 build is still available (make WRO_LIDAR=ORADAR for the Open Challenge, cmake -DWRO_LIDAR=ORADAR for the Obstacle Challenge). The emergency stop also sends the LiDAR stop frames so its motor does not keep spinning after the program ends.

3.5 Failure: the LiDAR could not see the walls (July 2026)

Symptom. After finishing the compact v4 chassis, the robot behaved as though it were blind - it was not detecting the track walls at all. **Cause.** The compact repackaging had placed the LiDAR high in the frame. The track walls are 100 mm tall, and the scanning plane was passing over them. **Fix.** In v5 we revised the central mount to bring the LiDAR down to roughly 60-80 mm above the floor while keeping the rear field of view clear for parking.

What we took from it. Sensor placement is a constraint imposed by the field, not a packaging variable. We treat the 100 mm wall height as a hard design input, and in v7 we went further: the whole chassis was lowered so the LiDAR scanning plane sits about 75 mm above the ground, 25 mm below the top of the walls.

3.6 Vision: OAK-D Lite → Camera Module 3 Wide + Hailo-10H → Hailo-8

The 2025 vehicle used an OAK-D Lite, an integrated stereo camera with onboard inference. For 2026 we split the job into a Raspberry Pi Camera Module 3 Wide for capture and a Hailo accelerator for inference.

We chose the split camera + accelerator instead of the integrated OAK-D Lite because it was lighter, drew less power, and inferred faster. The cost is that we own the whole capture, preprocess, infer and postprocess pipeline - which is also what let us train and deploy our own model (Section 4.8).

Design v6 used the Hailo-10H (AI HAT+ 2). Design v7 uses a **Hailo-8** (HailoRT 4.23). The same model was compiled for both chips and gives the same accuracy (mAP50 0.991 on the Hailo-10H, 0.990 on the Hailo-8) at **8 ms per frame** on the Hailo-8.

We chose the Hailo-8 instead of the Hailo-10H because our YOLOv8n model is light enough to run on the Hailo-8 at the same accuracy, and the Hailo-8 board is lighter and smaller, which reduces the overall mass and dimensions of the vehicle. For a single small detection model the Hailo-10H was more than we needed.

3.7 What we removed: ultrasonic sensors

Previous seasons carried ultrasonic rangefinders. The 2026 vehicles have none: the LiDAR measures the same distances better, and the ultrasonics cost mass, power, GPIO pins and mounting volume.

3.8 Sensor placement and calibration (design v7)

Item	Setting	Rationale
LiDAR height	Scanning plane below the 100 mm wall height	Section 3.5; chassis lowered in v7
Gyroscope bias	Measured for 5 s at start-up, robot still	On the bench, 2 s left ~0.5° of drift in 20 s; the error falls with the square root of the averaging time, so 5 s reduces it about 1.6×
Gyro zero	After the start button is pressed	Automatic, with no team input: the 5 s bias calibration and steering centring run on their own after power-on (rules 9.9, 11.7)
Yaw sign	Turning clockwise must decrease yaw	Same convention as the SPIKE Hub, so the mission code is unchanged
Steering center	Servo position 750 = wheels straight	The linkage is assembled with the servo at 750
Distance	Advance 1000 mm must travel 1000 mm; else scale kDiametroRuedaMm	Encoder-based distances
Chassis self-return rejection	Returns closer than 150 mm discarded	The robot's own body is inside the LiDAR's field of view

4. Software architecture and obstacle strategy

4.1 The migration to C/C++

The 2025 Python codebase became the bottleneck as soon as we attached a higher-rate camera and LiDAR. We rewrote the acquisition and control logic in C/C++, and the main control loop stabilised at more than 50 Hz.

We chose to rewrite instead of optimising the Python because the problem was the cost of the runtime itself under multi-threaded, high-rate sensor load. Optimising would have bought a constant factor; changing language changed the ceiling.

4.2 Threading model

The Obstacle Challenge binary (object_detection) runs cooperating threads: a Hailo pipeline thread (preprocess, inference and postprocess on camera frames, keeping a global traffic_lights struct current); a LiDAR reader thread filling a 360-slot distance buffer; and Obstacle_Challenge_Thread, the navigation state machine, which reads both and issues motion commands - to the SPIKE Hub in v6, and through the spike_compat layer to the Hiwonder board in v7. The Open Challenge is a separate, simpler binary (src/ondevice/) that never starts the camera or Hailo pipeline.

4.3 Wall detection: Select_Wall()

#	Stage	What it does
1	Angular windowing	A 70°-wide window centred on the requested side, with the innermost ±3° excluded
2	Chassis filtering	Returns closer than 150 mm are the robot's own body
3	Radius Outlier Removal	A point is dropped with fewer than 3 neighbours within 300 mm (15° padding outside the window)
4	Segmentation	Split where consecutive points jump by more than 60 mm, then at corners by Iterative End-Point Fit (40 mm)
5	Scoring	A segment qualifies with ≥ 6 points, ≥ 200 mm extent and RMS residual < 15 mm against its Total-Least-Squares line
6	Per-side tracking	Later frames prefer the same physical wall (within 300 mm and 20°)

Total-Least-Squares (PCA) has no bias toward any wall orientation. The 200 mm minimum extent is what rejects the 50 × 50 mm traffic-sign pillars; without it a pillar reads as a wall and the robot follows an obstacle. With the S2L the same pipeline receives 3,200 points per turn, more than the MS200k, so each wall segment has more points to fit.

4.4 Heading estimation - and what it is not

Heading comes primarily from the gyroscope - the SPIKE Hub's in v6, the Hiwonder board's IMU (20 Hz) in v7. LiDAR wall-orientation readings correct the heading at defined points in the manoeuvre logic. This is **not** sensor fusion: there is no complementary or Kalman filter. It is gyro-primary heading with situational LiDAR correction.

> **We chose wall re-squaring instead of a complementary filter because** the gyro's drift over a single 3-minute round is small, and re-squaring against a physical wall at every corner resets it to a ground truth that a filter would only have estimated.

New in v7: the IMU updates at 20 Hz while the LiDAR loops call the steering every 2 ms, so the derivative term of the heading PD only sees a change in about 1 of 25 calls. This is a known limitation of the new board; the wall re-squaring at every corner keeps it from accumulating into a heading error.

4.5 Closed-loop steering

Controller (v6 gains)	Kp	Kd
Camera-based cube following	150	100
Gyro-based wall following	2	300000

Controller (v7 gains, spike_compat)	Kp	Kd
Spike_Forward: advances that end on a LiDAR distance	0.05	0
Spike_Advance_For_Degrees / _distance: fixed-distance advances	0.02	0.05
Same, in reverse (parking)	0.003	0

The faster motor, the board's IMU and the S2L changed how the robot responds, so the thresholds and the PD gains were recalculated for v7. All gain sets were obtained by trial and error on the track: small adjustments to the values, run after run, until the behaviour was satisfactory. In v7 the PD output is scaled to the servo range by kPdADireccion = 0.133 × 200 / kDireccionRecorrido, which keeps the same sensitivity per degree of heading error as on the SPIKE Hub. No integral term is used: steady-state offset is not our failure mode, and an integrator accumulating during a cube-avoidance manoeuvre would fight the next correction.

4.6 Open Challenge strategy

The run is direction-agnostic: an initial LiDAR read determines which side is open (sentido), and the entire path is the sign-mirror of the other direction. The v6 constants were: outside/inside start split 800 mm; first straight 1350 / 300 mm; two Slope() reads 300 + 700 mm apart averaged into one heading correction; 80° corner turns; final approach bands 400 / 600 mm; final front threshold 1500 mm.

Changes in v7 (src/ondevice/main.cpp): the start decision uses three LiDAR readings that must agree within 30 mm (Lidar_Debounced_Read), so a single noisy reading cannot choose the wrong start box; the run is 12 corners (the first before the loop, then while (v < 11)) with corner turns re-tuned to 70-80° for the new drivetrain, and it ends with a single stop with the front wall at 1500 mm, the middle of the starting side.

4.7 Obstacle Challenge strategy

Direction is decided once from the initial side-wall reading (600 mm split). The unit of the run is one corner cycle - Corner_Case followed by Desicion - repeated 12 times (3 laps × 4 corners), with the 4th corner of each lap run in parking mode. Corner_Case advances to 1000 mm from the front wall, reads the pillar colour, advances a distance that depends on the colours of this and the previous pillar (600/400 mm or 990 mm; 650 mm in every other case) and turns 86°. Desicion re-squares the gyro against the outer wall and dispatches to the avoidance routines. Green means pass on the left, red on the right; this is fixed by the rules and does not mirror with the driving direction.

Parking. The camera does not locate the parking lot: the model keeps an xparking class only so that the magenta markers are never taken for pillars (those boxes, like any box scored below 0.75, are discarded). The robot knows where the lot is by counting sections: after 12 corner cycles it is back in the parking section, repositions according to the colour of the last pillar, and parks on LiDAR distances and gyro angles. Clockwise: advance to 150 mm from the front wall, reverse 2250 hub degrees (≈ 875 mm), then three reverse turns of 50°, 105° and 28° with a 400-degree (≈ 156 mm) reverse in between. Counter-clockwise: creep until the outer wall is ≤ 160 mm on the right, advance to 200 mm from the front wall, reverse 740 hub degrees (≈ 288 mm), then reverse turns of 52°, 100° and 28°. All values were tuned by hand on the track. In v7 the mission is the national mission with targeted substitutions marked "Hiwonder:" in the code, plus a LiDAR-guarded advance with a selectable brake type.

4.8 Vision model: training and measured performance

We train our own detector because the classes we care about - red and green traffic-sign pillars and the magenta parking markers, under competition lighting - are not COCO classes. Design v6 used a model trained on 501 images (two pillar classes; mAP50 0.986). For v7 we retrained on a much larger dataset.

Why we retrained. On the track the v6 model made three kinds of mistakes: it sometimes missed a pillar that was there, it took objects outside the track for pillars (a red broom read as a red pillar), and it sometimes saw a green pillar in an empty corner. Most of these failures happened in poor lighting. The new dataset adds photos from the robot's own camera in exactly those situations, including images with no pillars at all, so the model also learns when there is nothing to detect.

Source	Originals	Description	Validation
octubre	3,831	Photos from the car's own camera (800×600), labelled Oct 2026	5 complete recording sessions held out (477 images)
combinaditos	501	Phone photos with pillars of both colours	Photos c420 and above
roboflow_v3	5,022	Originals (320×320) of our previous Roboflow project	Its own split (495 test images)
Total	9,354	Roboflow wro-dataset-completo v1, exported 2026-10-18	

Only the training split is augmented, three versions per source image: 7,292 × 3 = 21,876 training images, plus 1,567 validation and 495 test originals = **23,938 images**. Pre-processing: auto-orientation and resize to 640×640 (stretch). Augmentation: horizontal flip (50 %), rotation ±5°, brightness ±20 %, exposure ±10 %, Gaussian blur 0-1 px and salt-and-pepper noise on 1 % of pixels - chosen to imitate the lighting changes and motion blur of an unknown venue. The model is YOLOv8n at 640×640, stretched with no letterbox to match the preprocessing on the car (Ultralytics 8.4.115, run h10_yolov8n_640_v4).

Because the dataset covers many different lighting conditions, the model rarely misses a pillar on the track; it fails consistently only in very poor lighting, close to darkness. The few errors we see on the track come from strong shadows.

> **We validated on held-out recording sessions instead of random frames because** consecutive video frames are nearly identical; a random split would leak near-copies of training images into validation and inflate the score.

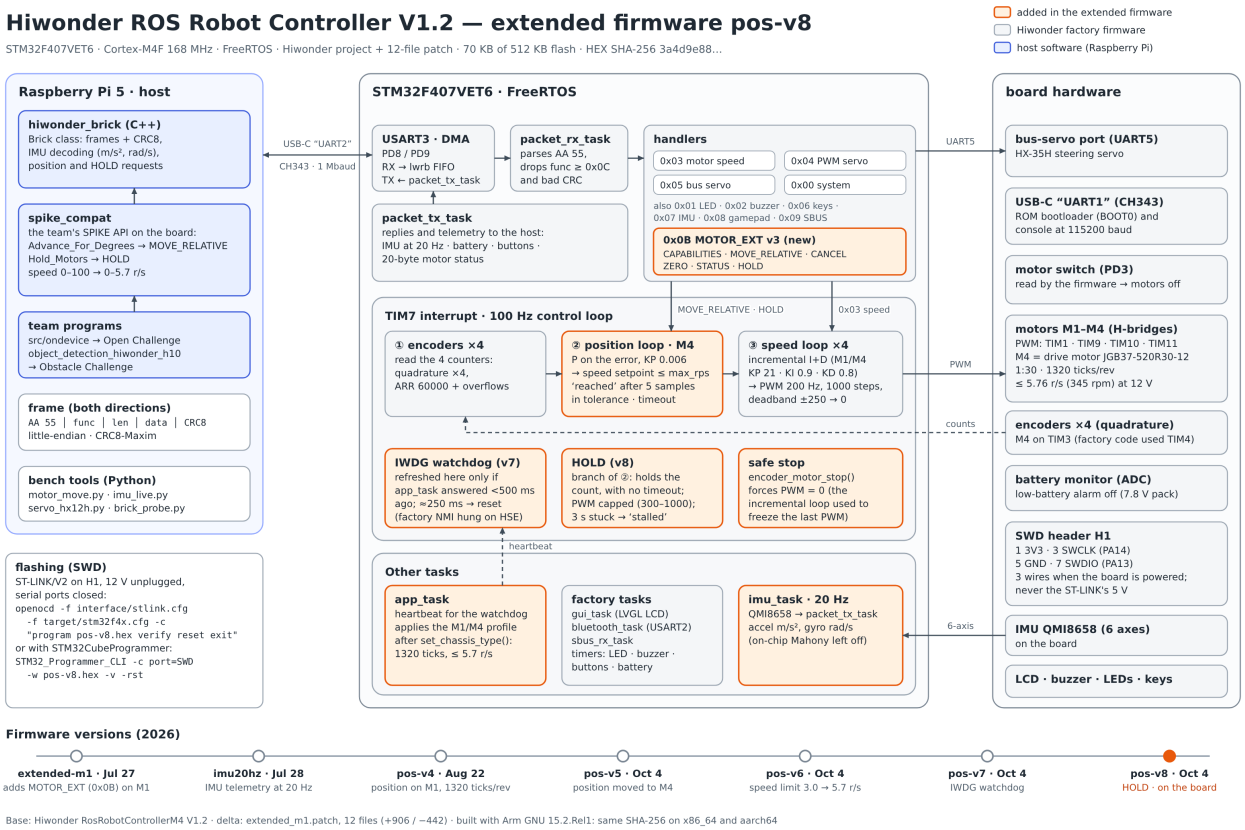
Test set	Metric	Result
octubre validation (477 images), Hailo-10H	mAP50	0.991
octubre validation (477 images), Hailo-8	mAP50	0.990
Same images, both chips	Correct Obstacle Challenge decision (colour → side)	99.2 %
combinaditos held-out (81 images, 275 pillars)	mAP50-95 / recall	0.875 / 0.982
Hailo-8 on the robot	Inference time	8 ms per frame
On the track, perception-only mode	Pillar colour / parking marker	96 % / 95 %

4.9 Hardware abstraction: the Hiwonder board behind a SPIKE-compatible layer

Why the hub had to go. The SPIKE Hub cannot drive the new JGB37 motor, so v7 needed a new controller that works with it and also replaces what the hub gave us, above all its integrated IMU for the heading. The Hiwonder board drives the encoder motor and the bus servo, has an on-board IMU, and its 5 V / 5 A output also powers the Raspberry Pi.

Hiwonder ROS Robot Controller (Hiwonder)	Value	How we use it
Microcontroller	STM32F407VET6, Cortex-M4 168 MHz, 512 KB flash	Our modified firmware pos-v8
Power input	DC 7-14 V	3S pack, 9.9-12.6 V
5 V output	Independent 5 V / 5 A, USB Type-C	Powers the Raspberry Pi 5
Encoder motors	4 channels, PID speed control	Port M4: JGB37 drive motor
Servos	4 PWM (5 V), 2 serial bus (battery voltage)	HX-35H steering servo, ID 1
IMU	On-board 6-axis IMU, QMI8658	Heading, 20 Hz
Communication	2 USB serial, CAN, SBUS, I2C, UART	USB serial to the Pi, 1 Mbaud
Controls	Main power switch, motor enable switch, 2 buttons, LED, buzzer	Main switch = hardware kill switch
Size, mass	85 × 60 mm, 39.5 g	-

Board firmware (pos-v8). The board runs Hiwonder's FreeRTOS firmware with our patch on top: extended_m1.patch changes 12 files of Hiwonder's official project (+906 / -442 lines). We added position control on the drive motor (a P loop on the encoder error that feeds the factory speed loop 100 times per second, so distances are measured on the board, 1320 ticks per output-shaft turn), HOLD (the motor keeps its position and comes back if pushed, like motor.HOLD on the SPIKE hub, and lets go after 3 s stalled), an IWDG watchdog that resets the board in about 250 ms if the firmware hangs, and smaller fixes (a stop forces PWM 0, the M4 encoder on its correct timer, IMU telemetry at 20 Hz, a 5.7 rev/s speed limit). The Pi uses these through one new command, MOTOR_EXT (function 0x0B). At start-up the program checks that the board answers it and stops before anything moves if it does not. The image, the patch, the build scripts, the protocol and the flashing steps are in firmware/hiwonder_pos-v8/ in the repository. Hiwonder's original source project (V1.2) and its factory recovery images are in firmware/hiwonder_original/, unmodified and credited to Hiwonder and the third-party libraries it uses.



Extended firmware pos-v8: host software on the Raspberry Pi 5, FreeRTOS tasks and the 100 Hz control loop on the STM32, and the board hardware. Orange blocks are our additions. Also in the repository as schemes/firmware/hiwonder_firmware_architecture.png.

Replacing the SPIKE Hub changed every motion function the missions call. Rewriting the missions would have meant re-tuning, from zero, the constants that placed 2nd at the national final, with less than eight weeks to the repository deadline.

> **We chose a compatibility layer instead of rewriting the missions because** it keeps the tuned mission logic and moves all hardware differences into one place that can be tested without the robot. hiwonder_brick implements the board protocol; spike_compat reproduces the hub semantics: Spike_Reset_Gyro receives decidegrees like the hub's reset_yaw(); speed 0-100 becomes rev/s in closed loop; Spike_Turn_For_Degrees turns with full steering like the hub's turn(); Spike_Advance_For_Degrees converts hub degrees to encoder counts and stops 40 counts early, then coasts, like the hub's ag(). All robot calibration lives in one file (hiwonder_config.h) shared by both challenges.

The translations are covered by offline tests against a fake board that records the commands it receives: **53 checks** for the Open Challenge layer and **31** for the Obstacle Challenge layer. The Open Challenge ran on the board on 2026-10-04 and the Obstacle Challenge on 2026-10-18.

4.10 Emergency stop

In the national edition the emergency stop was an **open, accepted risk**. Moving to the Hiwonder board made it urgent: its firmware has no command timeout (the IWDG watchdog we added only resets a hung firmware), so if the program dies while the motor is running, the motor keeps the last speed.

> **We chose a signal-based stop installed before anything moves** (parada_emergencia.cpp). It opens a second descriptor of the board's port with pre-built frames, so the signal handler only has to write them: four motors to COAST, servo torque off, LiDAR stop frames, then exit with code 130 without returning to the mission. It covers Ctrl+C, kill, closing the terminal or SSH session, Ctrl+Z and crashes. It does not cover kill -9 or a Raspberry Pi power loss. For those, the hardware kill switch is the Hiwonder board's power switch: it cuts the power to the motor, the servo and, through the board's 5 V output, the Raspberry Pi and every sensor.

5. Systems thinking: tradeoffs, risks and mitigations

5.1 Tradeoff register

#	Decision	Rejected alternative	Why	Cost accepted
1	Regular Ackermann	Double-Ackermann (v2)	Repeatability beat turning radius	Wider turning circle
2	Rear-wheel drive	All-wheel drive (v2)	Traction was never the limit	Less grip in a low-grip case
3	Clean-sheet 2026 frame	Retrofitting v2	All three problems were layout problems	Restarted the mechanical design
4	C/C++	Optimising Python	Needed a higher ceiling	Weeks with no new features
5	Oradar MS200k (v5-v6)	RPLidar S2L	Readings near walls and mass (later found to be our integration)	New driver; reversed in #14
6	Camera + Hailo	OAK-D Lite	Lighter, lower power, faster	We own the vision pipeline
7	No ultrasonics	Keep as redundancy	LiDAR measured it better	No independent backup
8	Gyro + wall re-squaring	Kalman / complementary filter	Wall gives ground truth	No continuous fused estimate
9	No integral term	Full PID	Offset is not our failure mode	Small offsets uncorrected
10	Section counting + LiDAR parking	Vision-detected parking	Position known from the run itself; no dependence on marker detection in the lot	Relies on every section being counted correctly
11	Hiwonder board + spike_compat (v7)	SPIKE Hub; rewrite missions	Speed and encoder control; keep tuned missions	A translation layer to maintain
12	JGB37 30:1 encoder motor (v7)	LEGO Large Angular Motor	1.7× speed, closed-loop distance	Heavier (152 g); 12 V supply needed
13	HX-35H bus servo (v7)	PWM hobby servo	Own position loop and feedback	More torque and mass than needed
14	Back to RPLidar S2L (v7)	Keep MS200k	Fixed integration; more points per turn	185 g
15	Single 3S pack (v7)	Separate Pi and motor packs	One battery, direct 12 V	All cells carry the peak current
16	New rear differential (v7)	Adapt the v6 differential	Fits the new chassis and motor; same 20:28 ratio	New mount to design and print
17	Modify v6 frame for v7	New clean sheet	The layout was right	Some packaging compromises
18	Signal-based emergency stop (v7)	Command timeout in the firmware	Works with any firmware; stops servo and LiDAR too	Not covering kill -9
19	Single vehicle, no spare	Build two	Budget; a spare is not allowed on the day	No fallback if it breaks in transit

5.2 Risk and failure-mode register

Risk	How it shows up	Mitigation	Status
Gyro drift over a run (≈ 1°/min measured)	Robot ends a corner mis-aimed; failures on lap 3	Re-square against the side wall at every corner; 5 s bias calibration	Mitigated
Pillar misread as wall	Robot follows an obstacle	200 mm minimum wall extent	Mitigated
Voltage sag under load	Sudden reset mid-run	Low-resistance cells; BMS; regulated 5 V / 5 A output for the Pi	Mitigated
Peak current (motor + servo stall)	Rail sag, Pi reset	Steering clamp avoids pushing against end stops; closed-loop speed control limits start-up current; low-resistance cells	Accepted
Speed set-point above motor capability	Speed changes as the battery drains	Effect measured: −9.7 % speed on a low battery; set-point 4.0 tested, −6.1 % (2.3, 6.1)	Characterised
LiDAR blind to walls	Total navigation failure	Chassis and mount kept low (3.5)	Resolved

Risk	How it shows up	Mitigation	Status
Program dies with motor running	Robot keeps driving	Emergency stop (4.10)	Resolved in v7
Steering past the mechanical limit	Wheel hits the chassis	Single clamp; start-up range check	Mitigated
Pillar colour misread (shadows, very poor light)	Wrong passing side	Dataset with many lighting conditions; 96 % on track	Mitigated
.hef / runtime version mismatch	Model fails to load	Versions pinned and documented	Documented
Wireless used during a round	Disqualification	Procedure below	Procedure defined
Sensor failure mid-run	Robot continues on stale data	No automatic fallback; start-up waits for the first LiDAR scan	Open, accepted

Wireless compliance procedure. During development we work over Wi-Fi SSH. At competition we connect over wired Ethernet for setup between rounds, disable the Raspberry Pi's Wi-Fi and Bluetooth radios, physically disconnect the Ethernet cable before the round, and start the vehicle with the single power switch and the single start button.

5.3 How the subsystems constrain each other

National season: reduce mass → steering torque drops → smaller steering motor → mass and volume freed → denser packaging → CG lowered → Ackermann stays effective → cornering more repeatable. And simultaneously, the same denser packaging → LiDAR pushed upward → above the 100 mm walls → total perception failure (3.5).

International season: faster motor → 12 V parts → one 3S pack replaces the X1203 and the SPIKE battery → mass freed → room for the 185 g S2L → better LiDAR. In parallel, the faster motor → firmware speed set above the motor's capability → speed now depends on battery charge (2.3); and the 3S pack → every cell carries the motor and servo peaks → the brownout risk of 3.2 returns in a new form. The change that gave us speed also re-opened two risks we had closed.

6. Testing

The national edition named our biggest methodological gap: changes were judged by watching the robot, not by measurement. For the international vehicle we set up a testing workflow in five levels, so each problem is caught at the cheapest level where it can appear:

Level	What runs	What it catches
1. Offline tests	make test (53 checks, Open Challenge layer) and reto2_test (31 checks, Obstacle Challenge layer) against a fake board that records every command	Wrong translations of the SPIKE API: units, signs, distances, stop conditions
2. Perception only	run_hiwonder_h8.sh --solo-percepcion: camera, Hailo and LiDAR run, the controller is not opened	Model, camera and LiDAR problems with zero risk to the robot
3. Wheels lifted	Yaw-sign, steering-side and distance checks (Section 3.8)	Calibration errors that would send the robot the wrong way
4. Slow track runs	WRO_MAX_RPS=1.5: same distances and angles at low speed	Logic errors in the missions
5. Competition speed	Full runs with the emergency stop ready	Timing, braking distance and repeatability

Both offline suites pass with zero failures, and every change to the hardware layer is checked against them before it reaches the robot. On the track, every attempt is logged - including the failed ones - with its date, challenge, driving direction, start position, pillar layout, speed setting, laps completed, parking result, time, and the cause of any failure. The success rate and the mean time of the completed runs are what we use to compare two versions of a change.

6.1 Component tests (2026-11-08)

Test	Method	Result
Encoder resolution	Count per output revolution	1320 counts, matching 11 PPR × 30:1 × 4
Distance	Advance 1000 mm, 10 repetitions, tape measure	Mean 1001.9 mm, std. dev. 11.1 mm, range 985-1018 mm
Gyro drift	Robot still after the 5 s calibration	0.5° at 30 s, 1.1° at 60 s, 2.3° at 120 s, 3.2° at 180 s
Wall detection	Select_Wall() on a straight at 0.4 m/s, 500 frames	S2L 492/500 (98.4 %); MS200k 481/500 (96.2 %)
Vision on track	Perception-only mode, 100 pillar and 20 parking approaches	Pillar colour 96/100 (96 %); parking marker 19/20 (95 %)
Steering servo	Bench	Center position 750 aligned with straight wheels
Offline test suites	make test, reto2_test	53 + 31 checks, 0 failures

LiDAR accuracy, S2L vs MS200k. Robot parallel to a wall, 50 readings per distance; error = mean measured − tape-measured distance.

True distance	S2L mean error	S2L std. dev.	MS200k mean error	MS200k std. dev.
100 mm	+2.1 mm	4.2 mm	+3.5 mm	5.1 mm
150 mm	+1.8 mm	4.6 mm	−2.8 mm	5.8 mm
300 mm	−3.5 mm	5.2 mm	+4.8 mm	6.9 mm
1000 mm	+8.4 mm	7.1 mm	−11.8 mm	9.8 mm
2500 mm	+22.6 mm	11.4 mm	−28.5 mm	16.3 mm

With its driver properly adapted, the S2L is as accurate as the MS200k at the short range where the July readings had failed, and better at every distance from 1 m, with lower noise throughout. Both are well inside the S2L datasheet accuracy of ±50 mm.

Speed vs battery (2 m straight, speed = 100):

Set-point	Battery	Time	Speed
kRpsVelocidad100 = 5.7	12.3 V	3.45 s	0.580 m/s
kRpsVelocidad100 = 5.7	10.7 V	3.82 s	0.524 m/s (−9.7 %)
kRpsVelocidad100 = 4.0	12.3 V	3.71 s	0.539 m/s
kRpsVelocidad100 = 4.0	10.7 V	3.95 s	0.506 m/s (−6.1 %)

At a similar battery voltage the measured speed (0.524 m/s at 10.7 V) is about 15 % below the 0.62 m/s calculated for 10.8 V (Section 2.3), and the full-battery speed (0.580 m/s at 12.3 V) is still below it, so we treat the calculated value as an upper bound: it only counts the torque needed to accelerate and to roll. The measured speed is still about 1.4 to 1.6 times the 0.365 m/s calculated for design v6. Lowering the set-point reduces the speed loss on a low battery from 9.7 % to 6.1 %, at the cost of about 7 % of top speed.

Reading the results. The distance spread (±11 mm) is the size of the errors the LiDAR wall re-squaring has to absorb at every corner. The gyro drifts about 1° per minute when still, so over a 3-minute round the heading would be off by about 3° without correction; this is why the gyro is re-squared against the side wall at every corner (Section 4.4). The four colour misclassifications in the vision test were all caused by shadows; the model only fails consistently in very poor lighting, close to darkness, because it was trained on images taken under many different lighting conditions (Section 4.8).

6.2 Open Challenge runs (2026-11-08)

#	Dir.	Start	Laps	Time	Result	Notes
1	CW	Outside	3	46.8 s	Pass	
2	CW	Inside	3	45.2 s	Pass	
3	CCW	Outside	3	49.1 s	Pass	
4	CCW	Inside	3	46.5 s	Pass	
5	CW	Outside	3	51.4 s	Pass	Minor drift on turn 2
6	CCW	Outside	3	48.2 s	Pass	
7	CW	Inside	3	45.9 s	Pass	
8	CCW	Inside	3	47.6 s	Pass	

#	Dir.	Start	Laps	Time	Result	Notes
9	CW	Outside	2	-	Fail	Clipped the inner wall on lap 3
10	CCW	Outside	3	48.3 s	Pass	

All runs at speed = 100. **9/10 completed (90 %)**, mean time of the completed runs **47.7 s** (range 45.2-51.4 s).

6.3 Obstacle Challenge runs (2026-11-08)

#	Dir.	Layout	Laps	Parking	Time	Result	Notes
1	CCW	A	3	Yes	118.5 s	Pass	
2	CW	A	3	Yes	122.0 s	Pass	
3	CCW	B	3	Yes	119.8 s	Pass	
4	CW	B	3	Yes	124.2 s	Pass	
5	CCW	C	3	Yes	117.5 s	Pass	
6	CW	C	3	Yes	121.0 s	Pass	
7	CCW	D	3	Yes	126.8 s	Pass	
8	CW	D	3	No	115.0 s	Partial	Overshot the parking bay line
9	CCW	E	2	No	-	Fail	Stuck on a red pillar on lap 3
10	CW	E	2	No	-	Partial	Clipped a green pillar on lap 3

All runs at speed = 100. **3 laps in 8/10 runs (80 %)**, parking completed in **7/10 (70 %)**, mean time of the complete runs **121.4 s** (range 117.5-126.8 s).

> **What the run logs tell us.** Every failure happened on **lap 3**, in both challenges, and two of the three Obstacle failures were on the same pillar layout (E). Combined with the gyro drift above, this points at accumulated heading and distance error late in the run rather than at a single bad manoeuvre. Our next changes target exactly that: the corner re-squaring on the third lap, the approach to the pillars of layout E, and the final parking approach (run 8).

6.4 National Final rounds (design v6)

The official rounds of the 2026 Mexico National Final are the largest test we have run so far: unknown field, unknown lighting, and no second attempt. Design v6 placed 2nd. The official-round videos of both challenges are linked in the repository README.



Preparing the vehicle at the field during the 2026 Mexico National Final. Photo: Geeks Academy.

7. Logbook

Date	Milestone	Section
2026-02-07	Repository created; season begins. Decision to rebuild rather than iterate the 2025 vehicle	1
2026-03-07 to 03-14	SPIKE and Raspberry Pi control functions ported to C; repository restructured	4.1
2026-03-21	KiCad schematics added; RPLidar S2L driver integrated	3.3
2026-04-18	Decision to revert to regular Ackermann for 2026	2.1
2026-05-09	v3 - clean-sheet chassis: RWD, regular Ackermann, battery and Pi low in the frame	2.5
2026-05-30	v4 - packaging refinement around the new layout	2.5
2026-06-13	Brownouts diagnosed; generic cells replaced with NCR18650B, X1203 integrated	3.2
2026-06-27	Battery pack reduced from 6 to 4 cells after re-running the power budget	3.1
2026-07-11	S2L short-range readings and mass; Oradar MS200k selected behind a compile-time switch	3.3
2026-07-25	LiDAR mounting-height failure found - scan plane above the 100 mm walls	3.5
2026-07-27 to 28	Hiwonder board: first extended firmware built with GCC (position-control command added to Hiwonder's FreeRTOS firmware); bench tests begin	4.9
2026-08-01	v5 - revised LiDAR mount, sensor lowered to 60-80 mm	2.5, 3.5
2026-08-08	OAK-D Lite replaced by Camera Module 3 Wide + Hailo-10H	3.6
2026-08-15	First vision model trained (501 images), mAP50 0.986	4.8
2026-08-22	Firmware pos-v4: position control validated on the bench with a new encoder motor (the first motor's encoder was faulty)	4.9
2026-08-29	v6 - final national geometry; vehicle photographed	2.5
2026-09-07	National documentation, BOM, flowcharts and repository organisation completed	8
2026-09-18 to 19	**Mexico National Final, Tijuana: 2nd place** (design v6)	-
2026-10-03	Program adapted to the national play field committed (release v1.0.0)	8
2026-10	Power system changed to one 3S NCR18650B pack with a BMS feeding the Hiwonder board (replaces the 4P pack, the X1203 and the SPIKE battery)	3.1
2026-10-04	Firmware pos-v5 to pos-v8 flashed: position control on M4, 5.7 rev/s limit, IWDG watchdog, HOLD	4.9

Date	Milestone	Section
2026-10-04	Open Challenge running on the Hiwonder board; HX-35H steering; emergency stop	4.9, 4.10
2026-10-11	Hailo-8 + RPLidar S2L support; S2L becomes the default; October dataset photos	3.4, 4.8
2026-10-18	Dataset wro-dataset-completo v1 exported; YOLOv8n v4 trained; Obstacle Challenge on the Hiwonder board	4.8, 4.9
2026-10 to 11	v7 chassis modified for the new parts and lowered; new rear differential	2.2, 2.4
2026-11-08	Test session on design v7: component tests, 10 Open and 10 Obstacle Challenge runs	6
2026-11-08	Hiwonder branch merged into main; releases v0.1.0-v2.0.0-alpha.1 published	8
2026-11-09 to 10	International documentation, commented code, firmware pos-v8 and design v7 print files added to the repository; releases v2.0.0-alpha.2 to v2.0.0	8

8. Reproducibility

Everything needed to rebuild this vehicle is in the public repository at github.com/alex309-duarte/WRO_Future_Engineers_Queretaro_CSS. Every milestone is a tagged GitHub release with release notes:

Release	Date	Vehicle state
v0.1.0	2026-06-20	First LEGO prototype, RPLidar S2L, C/C++
v0.2.0	2026-08-16	New chassis, Hailo-10H, Oradar MS200
v0.3.0	2026-09-13	Full documentation, first complete Obstacle Challenge
v1.0.0	2026-10-03	Design v6 - Mexico National Final, 2nd place
v2.0.0-alpha.1	2026-11-08	Design v7 development snapshot
v2.0.0-alpha.2	2026-11-09	International documentation snapshot
v2.0.0-alpha.3	2026-11-09	Source code commented in English
v2.0.0-beta.1	2026-11-09	Documentation feature-complete
v2.0.0-beta.2	2026-11-09	Review fixes, English program messages
v2.0.0-beta.3	2026-11-09	Component photos, new driving videos
v2.0.0-beta.4	2026-11-09	Board firmware (pos-v8 and Hiwonder original)
v2.0.0	2026-11-10	Design v7 - International Final

Content	Location
Control and vision source	src/ (v7: src/ondevice/ and src/cpp/object_detection_hiwonder_h10/)
Hiwonder board firmware	firmware/hiwonder_pos-v8/ (image, patch, build scripts, protocol, flashing steps); Hiwonder's original source and recovery images (credit: Hiwonder) in firmware/hiwonder_original/
Robot calibration	src/ondevice/hiwonder_config.h, src/cpp/object_detection_hiwonder_h10/reto2_config.h
CAD - SolidWorks parts, assemblies and STEP	models/cad/ (design v7: models/cad/v7_assembly/)
Printable STLs	models/cad/Stl/v7/ (12 parts, 16 pieces; printer and material in models/cad/Stl/README.md); design v6 parts in models/cad/Stl/
Vehicle photos (v7: *_v7.JPG) and design history photos	v-photos/, v-photos/Robot_More_Photos/Versions/
Wiring diagram	schemes/hardware/wiringSchematics_v7.png
Dataset and training results	other/dataset/, other/Training_model_and_results/; v7 dataset on Roboflow Universe (wro-dataset-completo)
Compiled models	src/HailoModels/ (v7: wro_h8_yolov8n_v4.hef)
Build, run and test instructions	README.md sections 6.9 and 9

Hardware interfaces (design v7)

Interface	Port	Rate
Hiwonder board	USB (CH343), detected automatically	1 Mbaud
RPLidar S2L	USB (UART adapter)	1 Mbaud
HX-35H servo	Serial bus on the Hiwonder board	-
Start button / LED	GPIO 4 / GPIO 3 (BCM)	-

Datasheets and references

Datasheets and product briefs that are published as PDF are also kept in the repository (other/datasheets/ and other/), so they stay available if a manufacturer moves or removes them.

Document	Link
WRO 2026 Future Engineers General Rules	Repository copy: other/WRO-2026-FE_GeneralRules.pdf Source: https://wro-association.org/wp-content/uploads/WRO-2026-Future-Engineers-Self-Driving-Cars-General-Rules.pdf
WRO 2026 Future Engineers Documentation Rubric	Repository copy: other/WRO-2026-FE_DocumentationRubric.pdf Source: https://wro-association.org/wp-content/uploads/WRO-2026-Future-Engineers-Documentation-Rubric.pdf
Hiwonder ROS Robot Controller: documentation	https://wiki.hiwonder.com/projects/ROS-Robot-Control-Board/en/latest/
Hiwonder JGB37-520 encoder motor (specifications)	https://www.hiwonder.com/products/hall-encoder-dc-geared-motor
Hiwonder HX-35H bus servo (specifications)	https://www.hiwonder.com/products/hx-35h
Slamtec RPLIDAR S2L	https://www.slamtec.com/en/s2
Slamtec RPLIDAR SDK	https://github.com/Slamtec/rplidar_sdk
Raspberry Pi AI HAT+ (Hailo-8) product brief	Repository copy: other/datasheets/RP-008133-DS-1-raspberry-pi-ai-hat-plus-product-brief.pdf Source: https://datasheets.raspberrypi.com/ai-hat-plus/raspberry-pi-ai-hat-plus-product-brief.pdf
USB Power Delivery on Raspberry Pi 5 (5 V / 5 A)	Repository copy: other/datasheets/RP-009856-WP-1-USB-Power-Delivery-on-Raspberry-Pi-5.pdf Source: https://pip-assets.raspberrypi.com/categories/685-app-notes-guides-whitepapers/documents/RP-009856-WP-1-USB%20Power%20delivery%20on%20Raspberry%20Pi%205.pdf
Raspberry Pi Camera Module 3 product brief	Repository copy: other/datasheets/RP-008151-DS-1-camera-module-3-product-brief.pdf Source: https://pip-assets.raspberrypi.com/categories/786-raspberry-pi-camera-module-3/documents/RP-008151-DS-1-camera-module-3-product-brief.pdf
Hailo-8 AI processor product brief	Repository copy: other/datasheets/Hailo-8-product-brief-rev3.26.pdf Source: https://hailo.ai/wp-content/uploads/2023/10/hailo-8-product-brief-rev3.26.pdf
Hailo AI Software Suite (Dataflow Compiler, HailoRT)	https://hailo.ai/products/hailo-software/hailo-ai-software-suite/
Panasonic NCR18650B datasheet	Repository copy: other/datasheets/Panasonic-NCR18650B-datasheet.pdf Source: https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/196/Panasonic-NCR18650B-datasheet.pdf
Ultralytics YOLOv8 documentation	https://docs.ultralytics.com/models/yolov8/
Dataset wro-dataset-completo (Roboflow Universe)	https://universe.roboflow.com/roboteam-x6iuc/wro-dataset-completo

9. What we would do differently - and what we did about it

Lesson from the national edition	What we did for v7
Check sensor geometry against the field before printing a chassis	The v7 chassis was lowered on purpose to keep the LiDAR plane below the walls
Specify power by internal resistance, not capacity	Kept the NCR18650B cells; peak-current analysis for the 3S pack (3.1)
Instrument the robot to log runs	Five-level testing workflow and a log of every run (Section 6)

New lessons from the international season:

- **Test an integration on its own before ruling out a component.** We dropped the S2L because of a rushed integration, then found it was the better sensor (3.4).
- **Check speed settings against the motor curve.** A speed setting above the motor's no-load speed silently turns closed-loop control into open loop (2.3).
- **Every hardware change can re-open a closed risk.** The single 3S pack brought back the peak-current problem we had solved in June (5.3).